

```
"""
```

```
breach_matcher.py – utilitários de filtragem do catálogo de breaches (legado).
```

Este módulo é usado pelo endpoint GET /breaches para decidir quais registros entram na resposta. Cada função opera sobre um "breach" no formato do feed da HIBP (um dict), por exemplo:

```
{
    "Name": "Adobe",
    "Domain": "adobe.com",
    "BreachDate": "2013-10-04",
    "AddedDate": "2013-12-04T00:00:00Z",
    "PwnCount": 152445165,
    "DataClasses": ["Email addresses", "Password hints", "Passwords"],
    "IsVerified": True,
    "IsSensitive": False,
    "IsSpamList": False,
}
```

NOTA: código legado herdado de outra squad. As docstrings descrevem como cada função DEVERIA se comportar – são a fonte da verdade do contrato.

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
# Slug de um breach: letras, dígitos, ponto e hífen. Não pode ser vazio.
```

```
_NAME_RE = re.compile(r"^[A-Za-z0-9.\-]+$")
```

```
def is_valid_breach_name(name: str) -> bool:
```

```
    """Retorna True se `name` é um slug de breach válido.
```

```
    Contrato: aceita apenas letras, dígitos, '.' e '-', e não pode ser vazio.
    Qualquer outro caractere (espaço, ';', aspas, etc.) torna o nome inválido.
```

```
    """
```

```
    if not name:
```

```
        return False
```

```
    return _NAME_RE.fullmatch(name) is not None
```

```
def domain_matches(breach: dict, query: str) -> bool:
```

```
    """Filtro de domínio: match PARCIAL e CASE-INSENSITIVE no campo `Domain`.
```

```

Ex.: query="adobe" casa com Domain="adobe.com".
      query="Dropbox" casa com Domain="dropbox.com".
Breaches sem domínio (Domain vazio/ausente) nunca casam.
"""

domain = (breach.get("Domain") or "").lower()
return query in domain


def data_class_matches(breach: dict, wanted: str) -> bool:
    """Retorna True se o breach expõe a classe de dados `wanted`.

    Contrato: comparação CASE-INSENSITIVE contra cada item de `DataClasses`.
    Ex.: wanted="passwords" casa com DataClasses=["Email addresses", "Passwords"]
    """
    wanted_norm = wanted.strip().lower()
    return any(wanted_norm == dc.strip().lower() for dc in breach.get("DataClasses"))


def within_breach_date(
    breach: dict,
    date_from: str | None = None,
    date_to: str | None = None,
) -> bool:
    """Filtra por `BreachDate` dentro da janela [date_from, date_to], INCLUSIVA.

    Datas no formato 'YYYY-MM-DD'. Limite None significa "sem limite" daquele lado.
    Ex.: date_from='2019-01-01', date_to='2019-12-31' deve INCLUIR um breach de
        BreachDate='2019-12-31'.
    """
    bd = breach.get("BreachDate") or ""
    if date_from and bd < date_from:
        return False
    if date_to and bd >= date_to:
        return False
    return True


def paginate(items: list, page: int, page_size: int) -> list:
    """Retorna a fatia da página `page` (1-indexada) com até `page_size` itens.

    Ex.: page=1, page_size=20 -> itens de índice 0..19.
        page=2, page_size=20 -> itens de índice 20..39.
    Paginando da primeira à última página, todos os itens devem aparecer.
    """
    start = (page - 1) * page_size
    end = start + page_size - 1

```

```

    return items[start:end]

def filter_breaches(
    breaches: list[dict],
    *,
    domain: str | None = None,
    data_class: str | None = None,
    breach_date_from: str | None = None,
    breach_date_to: str | None = None,
    min_pwn_count: int | None = None,
    max_pwn_count: int | None = None,
) -> list[dict]:
    """Aplica todos os filtros informados (semântica E / AND) e devolve os matches
    result = []
    for b in breaches:
        if domain is not None and not domain_matches(b, domain):
            continue
        if data_class is not None and not data_class_matches(b, data_class):
            continue
        if not within_breach_date(b, breach_date_from, breach_date_to):
            continue
        pwn = b.get("PwnCount", 0)
        if min_pwn_count is not None and pwn < min_pwn_count:
            continue
        if max_pwn_count is not None and pwn > max_pwn_count:
            continue
        result.append(b)
    return result

```